Network and System Administration

COSC4036

Chapter 2: Account & Security Administration, and Access Control

A Study Guide for Chapter 2

Credit Hours: 3 | ECTS: 5 | Contact Hrs: 2 | Lab Hrs: 3 | Tutorial Hrs: 1

Topics Covered in This Guide

Chapter Overview

Welcome to Chapter 2 of COSC4036 — Network and System Administration. This chapter covers one of the most critical responsibilities of a system administrator: controlling who has access to a system and precisely what they can do once inside.

Linux is a multi-user operating system — multiple users can be active simultaneously. This power demands careful management. Without proper account administration and access control, one careless or malicious user can compromise the entire system. Every topic in this chapter directly addresses that risk.

Chapter Learning Outcomes

Explain the three Linux user account types: root, human (user), and software accounts.
Distinguish between local and domain user accounts, and between security and distribution groups.
Describe the three Active Directory group scopes: Universal, Global, and Domain Local.
Create, modify, rename, and delete user accounts and groups using Linux commands.
Interpret all seven fields in /etc/passwd and all eight fields in /etc/shadow.
Explain the difference between local and LDAP account storage.
Use the finger and id commands to inspect user account information.
Configure password aging using chage and /etc/login.defs.
Interpret and modify file permissions using both symbolic and octal chmod notation.
Change file ownership using chown (with dot and colon notation) and chgrp.
Configure disk quotas: hard limits, soft limits, and grace periods.
Distinguish between DAC, RBAC, and MAC access control models.

2.1 User and Group Concepts, and User Private Group Scheme

Linux is a multi-user operating system, which means more than one user can use it at the same time. Every user and group on the system is uniquely identified by a numerical ID — the UID (User Identification number) and GID (Group Identification number) respectively.

Full user account information is stored in the /etc/passwd file. Hashed passwords are kept in /etc/shadow. Group information resides in /etc/group, and secure group account information in /etc/gshadow.

File	Contains	Access Level
/etc/passwd	Username, UID, GID, full name, home directory, login shell. Does NOT contain passwords.	World-readable
/etc/shadow	Encrypted password hashes and password aging fields.	Root only
/etc/group	Group names, GIDs, and group member lists.	World-readable
/etc/gshadow	Secure group account information including group passwords.	Root only

2.1.1 The Three Types of Linux User Accounts

Linux classifies every account into one of three types, each serving a distinct role:

Account Type	UID Range	Description	Security Implication
root	0 (always)	The superuser with unrestricted access to every file and command on the system. Home directory: /root.	Never use for everyday tasks. The kernel grants root power by checking UID=0, not the name.
Human (User) Accounts	Typically 1000+ (Red Hat: 500+)	Regular accounts for people who interact with the system interactively. Home under /home/username.	Passwords must be strong and periodically changed. Password aging applies here.
Software (System) Accounts	Below 1000	Accounts created for background services and daemons: www-data, mysql, daemon.	Should have no interactive login shell (/bin/false or /usr/sbin/nologin).

Why UID 0 = root Power (Always)

The Linux kernel does not grant privilege based on the username string 'root'.
It grants privilege based on whether the UID is exactly 0.
This means: renaming the root account does NOT remove its privileges.
It also means: NEVER create a second account with UID 0 — it would have full root access.
When in doubt, use: id root — it will always show uid=0.

2.1.2 Two Types of User Accounts

Account Type	Where Stored	Key Characteristics
Local User Account	On the local machine in /etc/passwd	Default for standalone systems. Simple to manage. Only works on that one machine.
Domain User Account	Active Directory (network directory service)	Stored centrally on a directory server. Same credentials work on every machine in the domain. Scalable for enterprises.

Account Name Rules — Important!

Account names can be between 1 and 20 characters in length.
Both letters and numbers are allowed.
Account names are NOT case-sensitive — alice and Alice refer to the same account.
The following special characters CANNOT appear in account names:
/ \ [ ] : ; | , + = * ? < > @

2.1.3 Two Types of Groups

Group Type	Purpose	How Permissions Work	Example
Security Group	Controls access to resources. Members share the same rights and permissions.	Assign permissions directly to the group. All members inherit them automatically.	A 'developers' group with read/write access to /srv/code.
Distribution Group	Used only for communication and information sharing — NOT for access control.	Cannot be assigned resource permissions. Used for sending bulk email.	A 'newsletter' group used by Microsoft Exchange to send emails to all members.

2.1.4 Active Directory Group Scopes

In Windows/Active Directory environments (which Linux admins frequently integrate with), groups have three possible scopes that define who can be a member and which resources they can access:

Scope	Who Can Be a Member	What Resources Can They Access	Best Use Case
Universal	Users and groups from ANY domain in the forest.	Resources in any domain.	Cross-domain access in large multi-domain organisations.
Global	Only users from the SAME domain.	Resources in any domain.	Grouping users with similar network access needs within one domain.
Domain Local	Users from any domain.	Resources ONLY within its own domain.	Granting access to local resources (printers, file shares) from members across domains.

Memory Aid: Global defines WHO, Domain Local defines WHERE

Global group = organises people (WHO gets access) — from the same domain
Domain Local = controls resources (WHERE access applies) — to local resources
Universal = spans everything — members and resources across all domains
Typical workflow: Users → Global Group → Universal Group → Domain Local Group → Resource

2.1.5 Using Groups Effectively

Groups enable administrators to manage access at scale. Key group-based tasks include:

- Assigning rights to a group account, authorising members to perform a specific task.
- Assigning permissions on shared resources to a group so all members access the resource in the same way.
- Distributing bulk email to all members of a distribution group.

When a user joins a team, you add them to the group — they instantly gain all the group's permissions. When they leave, you remove them — access revoked immediately. This is far more efficient than managing permissions for hundreds of individuals.

2.1.6 The User Private Group (UPG) Scheme

The UPG scheme is the default on modern Linux distributions (Red Hat, Fedora, CentOS, Ubuntu). It changes how new users are set up and makes the default permission model safer and more flexible.

How it works: When a new user (e.g., samojo) is created, a private group with the exact same name (samojo) is automatically created. The user's primary group is set to this private group, and they are its only member.

The benefit: The default umask is set to 002 (instead of the traditional 022). This grants full access to the owner and their group, while restricting others. Since the private group has only one member, the files remain secure. The arrangement also makes future collaborative sharing easy — just add a colleague to the private group.

umask	New File Permissions	New Directory Permissions	Context
022 (Traditional)	644 (rw-r--r--)	755 (rwxr-xr-x)	Group cannot write — conservative default
002 (UPG Default)	664 (rw-rw-r--)	775 (rwxrwxr-x)	Group can write — safe with UPG since group has one member
027 (Restrictive)	640 (rw-r-----)	750 (rwxr-x---)	Others have no access — used in secure environments

LAB EXERCISE 2.1-A: Exploring User Types, Groups, and the UPG Scheme

Objectives

- Identify the three types of Linux user accounts on a live system.
- Observe the UPG scheme creating a private group.
- Understand account name restrictions.

Steps

1. Run: cat /etc/passwd | head -15 — Identify system accounts (UID < 1000) vs. human accounts.
2. Run: id root — Confirm UID is always 0.

3. Run: id $USER — Note your UID, GID, and all group memberships.
4. Create a test user: sudo adduser testuser_01
5. Run: id testuser_01 — What GID was automatically assigned?
6. Run: cat /etc/group | grep testuser_01 — Was a private group created with the same name?
7. Run: umask — What is the default umask? Is it consistent with UPG?
8. Create a file as testuser_01: sudo su -c 'touch /tmp/upg_test.txt && ls -l /tmp/upg_test.txt' testuser_01
9. Attempt to create an invalid account: sudo useradd 'bad/name' — What error do you get?

Reflection Questions

Q1: If you rename the root account to 'superadmin', does it still have unrestricted access? Why?

Q2: With UPG and umask 002, a new file gets permissions 664 (rw-rw-r--). Explain why this is still secure.

Q3: What is the practical advantage of a Distribution Group over simply listing email addresses manually?

Answer here: _____

G Section Review Questions

1. Name the three types of Linux user accounts and describe each in one sentence.
Answer: _____

2. What is the key difference between a Security Group and a Distribution Group?

Answer: _____

3. List five characters that cannot be used in a Linux account name.

Answer: _____

4. Describe the three Active Directory group scopes and a use case for each.

Answer: _____

5. Explain the UPG scheme: what is created automatically, and why does it make umask 002 safe?

Answer: _____

2.2 User Administration, Modifying Accounts, and Group Administration

Managing user accounts is a fundamental and frequently performed task for any Linux system administrator. Linux provides a complete toolkit of commands for creating, modifying, and deleting both users and groups.

2.2.1 Overview of Core Administration Commands

Command	Category	Purpose
useradd	User	Adds a new user account to the system.
usermod	User	Modifies attributes of an existing account.
userdel	User	Deletes an account from the system.
chage	User	Views and modifies password expiry information.
passwd	User	Assigns or changes an account's password.
groupadd	Group	Creates a new group.
groupmod	Group	Modifies an existing group's attributes.
groupdel	Group	Removes an existing group.

2.2.2 Creating Users: useradd

The useradd command adds a new account with exactly the parameters you specify. It does not prompt interactively — ideal for scripts. Full syntax:

```
useradd -d homedir -g groupname -m -s shell -u userid accountname
```

Flag	Full Meaning	What It Does	Example
-d homedir	Home Directory	Specifies the path for the user's home directory.	-d /home/samojo
-g groupname	Primary Group	Sets the user's primary group.	-g developers
-m	Make Home	Creates the home directory if it does not already exist.	(always include for interactive users)
-s shell	Shell	Sets the default login shell.	-s /bin/bash
-u userid	User ID	Manually assigns a specific UID. System auto-assigns if omitted.	-u 1500

accountname	Username	The account name to create (final argument).	samojo

Complete example — creating user samojo:

```
sudo useradd -d /home/samojo -g developers -m -s /bin/bash -u 1500 samojo
sudo passwd samojo       # Set password separately — useradd does not prompt for one
```

What Happens When a User Is Created? A
home directory is created at /home/<username>
User info is written to /etc/passwd
Group membership is recorded in /etc/group
A hashed password entry is created in /etc/shadow
A mail spool file is created at /var/spool/mail/<username>
Default files from /etc/skel are copied into the new home directory

2.2.3 Modifying Users: usermod

The usermod command modifies attributes of an existing account. It accepts the same arguments as useradd, plus the -l option which renames the account login name.

Example — rename samojo to samojo00 and update the home directory:

```
sudo usermod -d /home/samojo00 -m -l samojo00 samojo
```

Flag	Purpose	Example
-l new_name	Rename the account (change login name).	usermod -l samojo00 samojo
-d new_home -m	Set a new home directory and move existing files there (-m copies them).	usermod -d /home/samojo00 -m samojo00
-aG group	Safely APPEND user to a supplementary group. Always use -a!	usermod -aG sudo samojo00
-s shell	Change the default login shell.	usermod -s /bin/zsh samojo00
-L	Lock the account (prepends ! to the shadow password hash).	usermod -L samojo00
-U	Unlock a previously locked account.	usermod -U samojo00

⚠ **CRITICAL: Always Use -aG, Never -G Alone**

WRONG: sudo usermod -G developers alice
Result: Alice's group list is REPLACED with just 'developers'.
Alice instantly loses sudo, audio, video, and ALL other group memberships!

CORRECT: sudo usermod -aG developers alice
The -a (append) flag adds 'developers' to Alice's existing group list.
All other memberships are preserved.
Rule: NEVER use -G without -a.

2.2.4 Deleting Users: userdel

The userdel command deletes a user account. It must be used with care — deletions are not easily reversed. There is only one key option:

```
sudo userdel -r samojo00          # Removes account AND home directory AND mail spool
sudo userdel samojo00             # Removes ONLY the account — home directory is preserved
```

Situation	Recommended Command	Reason
Employee permanently left	userdel -r username	Completely removes all traces: account, home dir (/home/username), and mail spool (/var/spool/mail/username).
Employee left, files needed for reference	userdel username (without -r)	Removes login access. Files in /home/username remain for backup or handover. You can delete the directory manually later.
Temporary account suspension	usermod -L username	Locks without deleting. Use usermod -U username to restore login access at any time.

2.2.5 Group Administration

Creating Groups: groupadd

Full groupadd syntax:

```
groupadd [-g gid [-o]] [-r] [-f] groupname
groupadd local_admin              # Simplest form — system auto-assigns GID
groupadd -g 484 sysadmin         # Create group sysadmin with GID 484
```

Flag	Meaning
-g GID	Manually specifies the numerical Group ID.
-o	Permits a non-unique GID (normally all GIDs must be unique).
-r	Creates a system group (assigned a GID from the system range, below regular user GIDs).
-f	Force — exits with success status even if the group already exists; suppresses error.
groupname	The actual name to give the new group.

Modifying Groups: groupmod

The groupmod command modifies an existing group. The most common uses are renaming the group (-n) or changing its GID (-g):

```
groupmod -n new_group_name old_group_name    # Rename a group
groupmod -n sysadmin local_admin             # Rename local_admin to sysadmin
groupmod -g 484 sysadmin                     # Change sysadmin's GID to 484
```

Deleting Groups: groupdel

To delete an existing group, use groupdel followed by the group name:

```
groupdel local_admin
```

> ⚠ **What groupdel Does and Does NOT Do**
> groupdel removes ONLY the group entry from /etc/group and /etc/gshadow.
> It does NOT delete any files that were owned by the group.
> Files previously owned by the deleted group will show a numeric GID in ls -l output.
> Those files remain fully accessible by their individual owners.
> Best practice: reassign or remove group-owned files BEFORE deleting the group.

LAB EXERCISE 2.2-A: User and Group Management: Create, Modify, and Delete

Objectives

- Use useradd with all key flags to create users.
- Rename a user with usermod -l.
- Create, modify, and delete groups.
- Observe the danger of -G without -a.

Steps

10. Create a group with GID 2000: sudo groupadd -g 2000 devteam
11. Create user samojo with home dir, bash shell, devteam group:
12. sudo useradd -d /home/samojo -g devteam -m -s /bin/bash samojo && sudo passwd samojo
13. Verify: id samojo and getent group devteam
14. Rename samojo to samojo00 and move home dir:
15. sudo usermod -d /home/samojo00 -m -l samojo samojo00
16. Verify: id samojo00 and ls /home/
17. Add samojo00 to sudo (safely): sudo usermod -aG sudo samojo00
18. Verify: groups samojo00 — confirm sudo is in the list
19. DANGER DEMO: sudo usermod -G devteam samojo00 (WITHOUT -a)
20. Run: groups samojo00 — What happened to the sudo membership?
21. Fix it: sudo usermod -aG sudo samojo00
22. Create second group: sudo groupadd webteam
23. Rename it: sudo groupmod -n webdevelopers webteam
24. Verify: getent group webdevelopers
25. Delete devteam: sudo groupdel devteam
26. Run: ls -la /home/samojo00/ — Note files with numeric GID where devteam used to be.
27. Delete samojo00 WITHOUT -r: sudo userdel samojo00 — then verify home dir still exists.
28. Delete testuser_01 WITH -r: sudo userdel -r testuser_01 — verify home dir is gone.

Reflection Questions

Q1: After running groupdel devteam, files in samojo00's home showed a number instead of a group name. What caused this and how would you fix it?

Q2: You need to disable an employee's account while HR investigates an incident, without losing any of their data. What is the safest command to use?

Q3: Why is it dangerous to run usermod -G groupname username without the -a flag?

Answer here: _____

2.3 Password Aging and Default User Files

Even a strong password becomes a liability if it is never changed. Password aging compels users to update their credentials periodically, limiting the window of exposure if a password has been stolen unnoticed.

2.3.1 Where Passwords and Default Files Live

Location	Purpose
/home/<username>	Default home directory for every human user account.
/root	Home directory for the root user — separate from /home for security.
/etc/shadow	Stores the encrypted password hash and all password aging fields.
/etc/login.defs	Defines system-wide defaults for password aging and UID/GID ranges.
/etc/skel	Skeleton directory. Files here are copied to a new user's home on account creation.
/var/spool/mail/<username>	Mail spool file created automatically when a user is added.

2.3.2 Inspecting Users: finger and id

The finger Command

The finger command displays detailed information about a specific user account. To view information about user chun, enter:

```
finger chun
```

Information Displayed	Description
Login	The username used to authenticate to the system.
Name	The user's full name (stored in the GECOS field of /etc/passwd).
Directory	The full path to the user's home directory.
Shell	The default shell that will be launched when the user logs in.
Last Login	The date, time, and location (terminal or IP) of the most recent login.

Note on finger

finger may not be installed by default. Install it with: sudo apt install finger

Many security-conscious servers disable finger because it reveals user information to remote callers.
Alternative (always available, no install): use id username or who

The id Command

The id command shows the UID, primary GID, and all supplementary group memberships for a given user:

```
id username
id chun          # View info for user chun
id              # View info for the currently logged-in user
```

Example output: uid=1001(chun) gid=1001(chun) groups=1001(chun),27(sudo),1003(devteam)

This tells you: UID is 1001, primary GID is 1001, and chun belongs to the sudo and devteam groups as well.

2.3.3 UID Values Across Distributions

Linux distributions use UIDs to identify users, but the numbering ranges differ between distributions:

Distribution	Regular User UIDs Start At	Notes
Ubuntu / Debian	1000	Most modern distributions use 1000 as the first regular user UID.
Red Hat / CentOS / Fedora	500 (older versions) or 1000 (RHEL 7+)	Older Red Hat systems started regular UIDs at 500. Modern RHEL uses 1000.
All distributions	0 (root)	root always has UID 0, regardless of distribution.
System accounts	Below the regular user minimum	System daemons and services use low UIDs (typically 1-999).

It is the UID that the operating system actually uses to control access to files and directories — not the username string. The username is only a human-readable label.

2.3.4 The chage Command — Password Aging

The chage (change age) command manages password aging for individual users. It reads and writes the aging fields in /etc/shadow:

Option	Full Name	Description	Example
-M days	Maximum	Maximum days a password is valid before the user MUST change it.	sudo chage -M 90 zewdu
-m days	Minimum	Minimum days that must pass before the user may change their password again.	sudo chage -m 7 zewdu
-W days	Warning	Days before expiry at which the user starts receiving login warnings.	sudo chage -W 14 zewdu
-I days	Inactive	Days of inactivity after expiry before the account is auto-locked.	sudo chage -I 30 zewdu
-E date	Expiry	Sets a hard account expiry date in YYYY-MM-DD format.	sudo chage -E 2026-12-31 zewdu
-l	List	Displays all current password aging settings for the user.	chage -l zewdu
-d 0	Force Reset	Forces a password change on the user's next login.	sudo chage -d 0 zewdu

Applying multiple policies at once (most efficient):

```
sudo chage -m 2 -M 90 -E 2026-06-30 -W 14 zewdu
```

2.3.5 System-Wide Defaults: /etc/login.defs

The /etc/login.defs file sets default password aging policies applied to all newly created accounts. Existing accounts are not retroactively changed.

```
sudo nano /etc/login.defs
```

Parameter	Description	Default
PASS_MAX_DAYS	Maximum days before a password must be changed.	99999 (unlimited) or 90 on hardened systems
PASS_MIN_DAYS	Minimum days required between password changes.	0 (no minimum) or 7 on hardened systems
PASS_WARN_AGE	Days before expiry to warn the user at login.	7
PASS_MIN_LEN	Minimum acceptable password length.	5 (recommended: 8+)

UID_MIN / UID_MAX	Range of UIDs assigned to regular users.	1000 / 60000
GID_MIN / GID_MAX	Range of GIDs assigned to regular groups.	1000 / 60000

LAB EXERCISE 2.3-A: Password Aging, finger, and id in Practice

Objectives

- Use finger and id to inspect user account information.
- Configure password aging policies with chage.
- Observe the effect of chage -d 0 (forced reset).

Steps

29. Install finger if needed: sudo apt install finger
30. Inspect your account: finger $USER — record Login, Name, Directory, Shell, Last Login.
31. Get your UID and groups: id $USER
32. Verify root's UID: id root — confirm it is always 0.
33. Create testuser_01 if needed: sudo adduser testuser_01
34. Check current aging: chage -l testuser_01
35. Apply a full aging policy:
36. sudo chage -M 90 -m 7 -W 14 -I 30 -E 2027-12-31 testuser_01
37. Verify: chage -l testuser_01 — confirm all six fields updated.
38. Force password reset: sudo chage -d 0 testuser_01
39. Switch to testuser_01: su - testuser_01 — Are you prompted to change password immediately?
40. Return: exit
41. Open /etc/login.defs: sudo nano /etc/login.defs
42. Find PASS_MAX_DAYS, PASS_MIN_DAYS, PASS_WARN_AGE. Record their current values.

Reflection Questions

Q1: What is the difference between setting aging via chage versus /etc/login.defs? Which takes effect for already-existing users?

Q2: Why is the -m (minimum days) option important? What specific attack does it help prevent?

Q3: Why might a security-conscious organisation remove the finger command from production servers?

Answer here: _____

2.4 Where Linux User Accounts Are Stored

When a Linux system is installed, administrators choose where user accounts will be stored. Understanding these storage options — and knowing how to read the account files — is essential for effective administration.

2.4.1 Authentication Storage Options

Method	Storage Location	Description	Best Suited For
Local	Files on local machine: /etc/passwd, /etc/shadow, /etc/group	The traditional default. Accounts stored in flat text files. Simple, always available without network access, but only works on that one machine.	Standalone servers, small deployments, lab environments.
LDAP (OpenLDAP)	Directory service on a network server	Newer and increasingly popular. Unlike flat files, a directory service is hierarchical — accounts can be organised by department, location, or function. A single set of credentials works across every machine in the organisation.	Enterprise environments. Universities. Any organisation with many computers and centralised IT.

Local vs. LDAP — A Practical Analogy

Local authentication is like a paper address book kept in a locked drawer — only usable on that one machine.

LDAP is like a shared contact database in the cloud — every machine in the building can query the same user accounts.

Universities and large enterprises almost universally use LDAP (or its Microsoft equivalent, Active Directory) so that one username and password works on every computer.

2.4.2 The /etc/passwd File — All Seven Fields

Every line in /etc/passwd represents one user. Fields are separated by colons. The format is:

```
Username:Password:UID:GID:Full_Name:Home_Directory:Default_Shell
```

A real example entry:

```
zewdu:x:1001:1001:Zewdu Tadesse:/home/zewdu:/bin/bash
```

Field #	Name	Description	Example
1	Username	The login name used when authenticating. The human-readable identifier.	zewdu
2	Password	Legacy field. An 'x' here means the actual password is stored in /etc/shadow. An empty field would mean no password (dangerous). Never stores the real password anymore.	x
3	UID	The numerical User ID. The kernel uses this number — not the username — to control file access.	1001
4	GID	The numerical Group ID of the user's primary default group.	1001
5	Full Name (GECOS)	Stores the user's full name and optionally other info (room, phone). Read by the finger command.	Zewdu Tadesse
6	Home Directory	Full path to the user's home directory.	/ home/zewdu
7	Default Shell	The shell program launched when the user logs in.	/ bin/bash

Why Was the Password Moved Out of /etc/passwd?

Originally, encrypted passwords were stored in /etc/passwd field 2.

/etc/passwd must be world-readable — many system utilities need to look up UIDs and usernames.

Storing password hashes in a world-readable file allowed anyone to copy and crack them offline.

Shadow passwords: moved the hash to /etc/shadow, readable only by root.

The 'x' in field 2 tells the system: 'look in /etc/shadow for the real password hash'.

2.4.3 The /etc/shadow File — All Eight Fields

The /etc/shadow file stores one line per user with eight colon-separated fields:

```
Username:Password:Last_Modified:Min_Days:Max_Days:Days_Warn:Disabled_Days:Expire
```

Field #	Name	Description	Example
1	Username	The login name — must match the corresponding /etc/passwd entry.	zewdu
2	Password	The hashed password (e.g., SHA-512 with salt). '!' at the start means the account is locked.	6salt$hash...
3	Last Modified	Number of days since 1 January 1970 (Unix epoch) when the password was last changed.	19500
4	Min Days	Minimum days required before the user can change their password.	7
5	Max Days	Maximum days the password is valid before it must be changed.	90
6	Days Warn	Days before expiry that the user is warned at login.	14
7	Disabled Days	Days of inactivity after password expiry before the account is automatically locked.	30
8	Expire	Absolute account expiry date as days since the Unix epoch.	20000

LAB EXERCISE 2.4-A: Reading and Interpreting the Account Files

Objectives

- Decode all seven /etc/passwd fields for multiple accounts.
- Understand the /etc/shadow 8-field structure.
- Verify that /etc/shadow access is restricted to root.

Steps

43. View the first 10 lines: cat /etc/passwd | head -10
44. For the first 5 entries, identify: Username, UID, GID, Home Directory, Shell.
45. Find a system account (UID < 1000). What shell is assigned? Why?
46. Check root: grep '^root' /etc/passwd — Verify both UID and GID are 0.

47. Attempt to read shadow as normal user: cat /etc/shadow — What error appears?
48. Read shadow as root: sudo cat /etc/shadow | head -5 — Observe the hash in field 2.
49. Find testuser_01's shadow entry: sudo grep testuser_01 /etc/shadow
50. Count the colons in that line — there should be exactly 7 (separating 8 fields).
51. Identify each field and match it to the table above.
52. Try: sudo usermod -L testuser_01 then: sudo grep testuser_01 /etc/shadow
53. Has the ! appeared in field 2? What does this mean?
54. Unlock: sudo usermod -U testuser_01 — confirm ! is removed.

Reflection Questions

Q1: Why does /etc/passwd need to be world-readable? What risk does this create?

Q2: What does it mean when the password field in /etc/shadow starts with '!'? How did it get there?

Q3: The Last_Modified field contains the number 19500. What date does this represent and why does Linux count days from January 1, 1970?

Answer here: _____

2.5 Controlling Access to Files and Permissions

Permissions are the core mechanism for operating system protection. They ensure users do not misuse system resources — CPU, memory, network, partitions, directories, and files. Permissions specify who can access a file or directory and exactly which types of access are permitted.

Linux controls permissions at three levels — commonly called the UGO model:

Level	Symbol	Who It Covers
Owner (User)	u	The individual user who owns the file. Usually the person who created it.
Group	g	All members of the group assigned to the file share the group permissions.
Others (World)	o	Every user who is not the file owner and not in the file's group.

2.5.1 The Three Permission Types

Permission	Symbol	Octal Value	Effect on a FILE	Effect on a DIRECTORY
Read	r	4	View or copy file contents (cat, less, cp).	List directory contents (ls).
Write	w	2	Modify or delete the file.	Create, rename, or delete files within the directory (requires execute too).
Execute	x	1	Run the file as a program or script.	Enter the directory (cd) and access its contents.
None	-	0	No access of this type.	No access of this type.

Critical Rule: Deleting Requires DIRECTORY Write Permission

To delete a file, you do NOT need write permission on the file itself.
You need write permission on the DIRECTORY that contains the file.
Reasoning: a directory is essentially a list of filenames. Deleting removes a name from that list.
Modifying the list requires write access to the directory.
This means: even if a file has permissions 000, the owner of the directory can still delete it.

2.5.2 Reading Permission Strings with ls -l

The ls -l command shows file permissions as a 10-character string. Example:

```
$ ls -l file1.txt
-rwxr-xr-x  1  root  root  316848  Feb 27 2017  file1.txt
```

Position(s)	Character(s)	Meaning
1	-	File type: '-' = regular file, 'd' = directory, 'l' = symbolic link, 'b' = block device
2-4	rwx	Owner permissions: root (the owner) can Read, Write, and Execute.
5-7	r-x	Group permissions: group 'root' members can Read and Execute, but NOT Write.
8-10	r-x	Others permissions: all other users can Read and Execute, but NOT Write.

Reading this example in plain English:

- The file file1.txt is owned by user 'root'.
- The owner (root) has the right to read, write, and execute the file.
- The file is owned by group 'root'. Group members can read and execute but cannot write.
- Everybody else can also read and execute the file but cannot write.

2.5.3 Changing Permissions: chmod

The chmod (change mode) command modifies permissions. It supports three methods.

Method 1: Symbolic Notation with + and - (Relative Changes)

Use class letters (u, g, o, a=all), operators (+ to add, - to remove), and permission letters (r, w, x). Best when you want to change specific bits without touching others:

```
chmod g-w,o-r file1.txt          # Remove write from group AND read from others
chmod u+x script.sh               # Add execute for owner only
chmod a+x deploy.sh               # Add execute for user, group, AND others
```

Method 2: Symbolic Notation with = (Exact Assignment)

The = operator sets permissions to exactly what you specify, completely overriding existing bits:

```
chmod u=rwx,g=r,o= file1.txt    # Owner: full; Group: read-only; Others: no
access
chmod g=rw shared_doc.txt       # Group: exactly rw (no execute)
chmod u=rwx,g-w+x,o-r file1.txt  # Combine = and + and - in one command
```

Method 3: Octal (Numeric) Notation

Each permission triplet (user, group, others) is represented by a digit: Read=4, Write=2, Execute=1. Add the values for the permissions you want.

Permission Set	Calculation	Digit	Symbolic
Read + Write + Execute	4+2+1	7	rwx
Read + Write	4+2+0	6	rw-
Read + Execute	4+0+1	5	r-x
Read only	4+0+0	4	r--
Write only	0+2+0	2	-w-
No access	0+0+0	0	---

Example: owner=rwx(7), group=r-x(5), others=no access(0) → chmod 750 file1.txt

```
chmod 750 file1.txt
```

Octal	Symbolic	Standard Use Case
755	rwxr-xr-x	Directories and executable scripts — others can read and enter
644	rw-r--r--	Regular files — owner can edit, everyone else reads
600	rw-------	Private files — SSH keys, personal config
640	rw-r-----	Config files group can read, others cannot see
750	rwxr-x---	Programs for owner and group, hidden from others
777	rwxrwxrwx	Fully shared — use with extreme caution on servers

```
chmod -R 644 /var/www/html/      # Recursively set all files in directory to
644
```

LAB EXERCISE 2.5-A: chmod: Decode and Apply All Three Methods

Objectives

- Decode permission strings from ls -l output.
- Apply all three chmod methods and observe results.
- Practice recursive permission changes.

Steps

55. Create test files: touch ~/f1.txt ~/f2.txt ~/script.sh && mkdir ~/testdir
56. Check initial permissions: ls -l ~/f1.txt ~/script.sh ~/testdir
57. METHOD 1 (+ and -): Remove group write and others read from f1.txt:

58. chmod g-w,o-r ~/f1.txt — Verify: ls -l ~/f1.txt
59. Add execute for owner on script.sh: chmod u+x ~/script.sh — Verify: ls -l
60. METHOD 2 (=): Set f2.txt to exactly rwx for user, r for group, nothing for others:
61. chmod u=rwx,g=r,o= ~/f2.txt — Verify: ls -l ~/f2.txt
62. METHOD 3 (octal): chmod 755 ~/script.sh — Verify: ls -l
63. chmod 644 ~/f1.txt — Verify: ls -l
64. chmod 600 ~/f2.txt — Verify: ls -l
65. Recursive: mkdir -p ~/testdir/subdir && touch ~/testdir/sub.txt ~/testdir/subdir/deep.txt
66. chmod -R 750 ~/testdir/ — Verify: ls -lR ~/testdir/
67. DECODE these permission strings (fill in symbolic, octal, and meanings):
68. -rw-r--r-- Octal=____ Owner=____ Group=____ Others=____
69. drwxrwx--- Octal=____ Owner=____ Group=____ Others=____
70. -r-xr-xr-x Octal=____ Owner=____ Group=____ Others=____

Reflection Questions

Q1: A file has permissions -rwxr-x---. The file's group member tries to delete it. Can they? Explain why, referring to the correct permission concept.

Q2: What is the octal value for rwxr-x--x? Show the complete calculation for each digit.

Q3: When would you choose Method 1 (relative) over Method 3 (octal)?

Answer here: _____

2.6 Managing File Ownership

Every file and directory is associated with exactly one user owner and one group owner. The chown and chgrp commands allow administrators to change these assignments.

Who Is Allowed to Change Ownership?

To change the USER owner: you must be logged in as root (or use sudo).

To change the GROUP owner: you must be root OR the current file owner — and you can only assign a group you belong to.

These restrictions exist to prevent users from circumventing disk quotas or permission barriers by transferring files.

2.6.1 chown — Change User and/or Group Owner

The chown (change owner) command changes the user or group that owns a file or directory.

Syntax:

```
chown user.group file_or_directory
chown user:group file_or_directory    # Colon notation also works
```

Command	What It Changes	Example
chown ncth1 /tmp/myfile.txt	Changes the user owner to ncth1.	sudo chown ncth1 /tmp/myfile.txt
chown .users /tmp/myfile.txt	Changes group owner to 'users'. The dot (.) before the name means group.	sudo chown .users /tmp/myfile.txt
chown student.users /tmp/myfile.txt	Changes BOTH user owner (student) AND group owner (users) at once.	sudo chown student.users /tmp/myfile.txt
chown -R user:group directory/	Recursively changes ownership of an entire directory tree.	sudo chown -R www-data:www-data /var/www/html/

Dot Notation vs. Colon Notation

chown ncth1 file — changes USER owner to ncth1 only

chown .users file — the dot (.) signals that 'users' is a GROUP name

chown student.users file — changes user to 'student' AND group to 'users'

Modern Linux also accepts colon: chown student:users file — identical result

Both notations are valid; the colon form appears more frequently in modern documentation.

You can use -R option with chown to change ownership on many files at once recursively.

2.6.2 chgrp — Change Group Owner Only

The chgrp (change group) command is an older, simpler tool that changes only the group owner of a file or directory:

```
chgrp group file_or_directory
chgrp student /tmp/newfile.txt      # Change group of newfile.txt to 'student'
chgrp -R webteam /srv/website/     # Recursively change group ownership
```

Note: chgrp student /tmp/newfile.txt is functionally equivalent to chown :student /tmp/newfile.txt — use whichever reads more naturally in context.

LAB EXERCISE 2.6-A: Practising File Ownership with chown and chgrp

Objectives

- Use chown with dot and colon notation.
- Verify root-only restriction on user owner changes.
- Apply recursive ownership to a directory.

Steps

71. Create a test file: touch ~/ownership_test.txt
72. Check current owner: ls -l ~/ownership_test.txt
73. Attempt WITHOUT sudo: chown root ~/ownership_test.txt — What error appears?
74. Use sudo: sudo chown root ~/ownership_test.txt — Verify owner changed.
75. Try to delete as normal user: rm ~/ownership_test.txt — Can you?
76. Change owner back (dot notation): sudo chown $USER ~/ownership_test.txt
77. Change ONLY the group (dot notation): sudo chown .$(id -gn) ~/ownership_test.txt
78. Verify both user and group: ls -l ~/ownership_test.txt
79. Create shared directory: sudo mkdir /srv/shared_project
80. Create group: sudo groupadd projteam
81. Recursively change ownership: sudo chown -R $USER:projteam /srv/shared_project/
82. Verify: ls -la /srv/shared_project/
83. Use chgrp alternative: sudo chgrp projteam /srv/shared_project/
84. Create a test file inside: sudo su -c 'touch /srv/shared_project/test.txt' $USER
85. Run: ls -l /srv/shared_project/test.txt — Note the ownership values.

Reflection Questions

Q1: What is the functional difference between chown .groupname file and chown username.groupname file?

Q2: If a file is owned by root and you (a sudo user) have write permission on it via group membership, can you delete it without sudo? Why or why not?

Q3: You accidentally ran sudo chown -R nobody:nobody /etc/. How serious is this? What would immediately break?

Answer here: _____

2.7 Managing Disk Quotas

On a shared Linux system — especially file servers used by many users — a single person or runaway process can fill an entire partition, crashing the system or preventing everyone else from saving work. Disk quotas are the proactive defence.

Implementing a disk quota prevents users or groups from using too much storage space. This is very useful on file servers where many users connect and store data — it ensures no single user can monopolise storage and disrupt the entire server. System administrators should always enforce quota limits on storage, maximum processes, and open file counts.

2.7.1 The Three Core Quota Concepts

Concept	Definition	What Happens When Reached
Hard Limit	The absolute maximum amount of disk space a user or group may use.	Write operations FAIL immediately. No further data can be written.
Soft Limit	A warning threshold below the hard limit. Designed to alert users before they hit the hard limit.	Can be temporarily exceeded during the grace period. User is warned at each login.
Grace Period	The time window during which the soft limit may be exceeded. Configurable in seconds, minutes, hours, days, weeks, or months.	Once the grace period expires without the user reducing usage, the soft limit is enforced as a hard limit until usage drops below it.

Practical Example: Soft vs. Hard

Student has soft=900MB, hard=1000MB, grace=7 days.

Student uploads 950MB → exceeds soft limit. Warning appears at next login.

7 days pass with no reduction → soft limit now enforced like hard limit.

Student tries to upload even 1 more byte → write fails (soft limit = hard limit now).

Student tries to upload 100MB more on day 1 → exceeds 1000MB hard limit → write fails immediately.

2.7.2 Implementing Disk Quotas — All Five Steps

Step 1: Install the Quota Package

```
sudo apt-get install quota          # Debian/Ubuntu
sudo yum install quota              # CentOS/RHEL
```

Step 2: Edit /etc/fstab to Enable Quotas

Edit /etc/fstab and add usrquota and/or grpquota to the mount options of the target filesystem:

```
sudo nano /etc/fstab
```

Example /etc/fstab before and after enabling user quotas:

```
# Example /etc/fstab with quota options:
LABEL=/1     /              ext3   defaults                          1 1
LABEL=/boot    /boot        ext3   defaults                          1 2
LABEL=/users    /users     ext3    exec,dev,suid,rw,usrquota         1 2
LABEL=/var     /var          ext3   defaults                         1 2
LABEL=SWAP     swap         swap   defaults                         0 0
```

To enable both user AND group quotas:

```
/dev/sda1  /home  ext4  defaults,usrquota,grpquota  0 2
```

Step 3: Remount the Filesystem

Remount so the kernel recognises the new quota options from /etc/fstab:

```
umount /users; mount /users       # Unmount then remount
mount -o remount /home            # Alternative: remount without unmounting
```

Step 4: Create the Quota Database Files

Use quotacheck to scan the filesystem and create the quota database (aquota.user and aquota.group):

```
quotacheck -cu /users             # -c: create files, -u: user quotas
quotacheck -cug /home             # -u: user quotas, -g: group quotas
```

Then enable enforcement:

```
quotaon /users                    # Enable quota enforcement on /users
quotaon -avug                     # Enable on all filesystems in /etc/fstab
with quota options
```

Step 5: Assign Quotas Per User with edquota

Use edquota to set specific limits for each user:

```
edquota -u web_cc                # Open quota editor for user web cc (uid
524)
```

The editor displays a table like this:

```
Filesystem     blocks     soft       hard       inodes     soft     hard
/dev/sdb1      988612     1024000    1075200    7862       0        0
```

Column	Description
Filesystem	The partition where the quota applies.
blocks (current)	Current disk usage in 1KB blocks. Auto-maintained — do NOT edit.
soft (blocks)	The soft limit in 1KB blocks. Set your warning threshold here.
hard (blocks)	The hard limit in 1KB blocks. Absolute maximum — cannot be exceeded.
inodes (current)	Current number of files. Auto-maintained — do NOT edit.
soft (inodes)	Soft limit on number of files. 0 = no limit on file count.
hard (inodes)	Hard limit on number of files. 0 = no limit on file count.

Inspecting Quota Status

```
quota -u web_cc                  # View quota and usage for a specific user
repquota /home                   # Full report: all users' quota usage on
/home
repquota -a                      # Full report for ALL filesystems with
quotas
```

LAB EXERCISE 2.7-A: Disk Quota Setup and Testing

Objectives

- Configure disk quotas on a test filesystem.
- Set soft and hard limits.
- Test enforcement by exceeding the soft then hard limit.
- Read and interpret repquota output.

Steps

86. Check quota tools: which quotacheck edquota repquota
87. Install if needed: sudo apt-get install quota
88. Create a test loop filesystem (safe for lab environments):
89. sudo dd if=/dev/zero of=/tmp/quota_test.img bs=1M count=100
90. sudo mkfs.ext4 /tmp/quota_test.img

91. sudo mkdir /mnt/quotalab
92. sudo mount -o loop,usrquota /tmp/quota_test.img /mnt/quotalab
93. Initialise database: sudo quotacheck -cum /mnt/quotalab
94. Enable enforcement: sudo quotaon /mnt/quotalab
95. Ensure testuser_01 exists: id testuser_01
96. Set limits: sudo edquota testuser_01
97. Set block soft=5120 (5MB), block hard=10240 (10MB)
98. Verify: sudo quota -u testuser_01
99. Test enforcement as testuser_01:
100. sudo su -c 'dd if=/dev/zero of=/mnt/quotalab/f1.img bs=1M count=6' testuser_01
101. (Should warn — exceeded soft limit)
102. sudo su -c 'dd if=/dev/zero of=/mnt/quotalab/f2.img bs=1M count=6' testuser_01
103. (Should fail — exceeded hard limit)
104. Report: sudo repquota /mnt/quotalab
105. Users over soft limit are marked with '+'. Interpret the output.

Reflection Questions

Q1: What is the difference between a block quota and an inode quota? Give a scenario where a user hits the inode limit but not the block limit.

Q2: If the grace period expires while a user is over their soft limit, what exactly changes about enforcement?

Q3: Why should every system administrator enforce disk quotas even on systems with seemingly plenty of free space?

Answer here: _____

G Section Review Questions

1. Describe the three quota concepts: hard limit, soft limit, and grace period. Answer: _____

2. List all five steps required to implement disk quotas on a Linux system. Answer: _____

3. What command do you use to view the quota usage of a specific user? Answer: _____

4. In the edquota editor, what does a value of 0 mean for both soft and hard inode limits? Answer: _____

5. Why must you run quotacheck BEFORE running quotaon? Answer: _____

Appendix: Access Control Models — DAC, RBAC, MAC

Access control is the framework underlying everything covered in this chapter. It governs how the system decides whether to grant or deny any request to access a resource.

A.1 The Four Access Control Components

Every access control decision involves exactly four components:

Component	Role (the 'question' it answers)	Example
Subject	WHO is requesting access?	User alice, the Apache web process, a backup script.
Object (Objective)	WHAT resource is being accessed?	File /etc/passwd, a database table, a physical printer.
Action (Operation)	WHAT action is being attempted?	Read, Write, Execute, Delete, Create.
Policy	What RULES govern the access decision?	DAC chmod bits, RBAC role assignments, MAC security labels.

A.2 The Major Access Control Models

Model	Full Name	Core Principle	Who Controls Access	Linux Implementation
DAC	Discretionary Access Control	The resource owner decides who can access their own files and what they can do.	The file owner	chmod and chown — the standard Linux permission system you have studied in this chapter.
RBAC	Role-Based Access Control	Permissions are assigned to job roles. Users inherit permissions by belonging to a role.	The administrator (via role assignments)	Linux groups — adding users to a 'developers' or 'sudo' group is role-based access control.
MAC	Mandatory Access Control	A central authority assigns security classification labels. The OS enforces them. Users cannot override.	The operating system	SELinux (Security-Enhanced Linux) on Red Hat/CentOS, AppArmor on Ubuntu.
PAM	Privileged Access Management	Special controls, monitoring, and audit	The security/IT team	sudo with logging, fine-grained

		logging for high-privilege administrator accounts.		NOPASSWD rules, privileged session recording.

Which Model Does Standard Linux Use?

Linux implements DAC (Discretionary Access Control) by default via chmod/chown.

RBAC is partially present via Linux groups — each group is effectively a 'role'.

MAC is available but optional via SELinux (Red Hat/CentOS) or AppArmor (Ubuntu/Debian).

Most production servers use: DAC as the base layer + RBAC-style group management + optionally MAC for critical services.

Capstone Lab: Securing a Multi-User Linux Server

This capstone lab integrates every concept from Chapter 2. You will configure a complete, realistic multi-user environment for a technology company.

Scenario: NovaTech Solutions

You are the system administrator for NovaTech Solutions.

Requirements:

Three teams: developers, analysts, managers — each with its own shared directory.

developers: read/write to dev_work. analysts: read-only to analysis. managers: full access.

All passwords expire every 60 days, minimum 3 days between changes, 7-day warning.

Each developer is limited to 500MB of storage.

New user default umask is 027.

All account actions must be recorded in /root/setup_log.txt for auditing.

TASK 1: Group and User Setup

106. Create groups with specific GIDs: groupadd -g 3001 developers / groupadd -g 3002 analysts / groupadd -g 3003 managers
107. Create users: dev1, dev2 → developers; ana1 → analysts; mgr1 → managers
108. Grant mgr1 sudo: usermod -aG sudo mgr1
109. Verify all: for u in dev1 dev2 ana1 mgr1; do id $u; done

TASK 2: Directory Structure and Permissions

110. Create: sudo mkdir -p /srv/novatech/{dev_work,analysis,management}
111. Set ownership: sudo chown root:developers /srv/novatech/dev_work (repeat for each)
112. Set permissions: dev_work → 770, analysis → 750, management → 770
113. Test: su - dev1 and attempt writing to each directory. Document all results.

TASK 3: Password Aging Policy

114. Apply to all users: for u in dev1 dev2 ana1 mgr1; do sudo chage -M 60 -m 3 -W 7 -I 30 $u; done
115. Force password reset: for u in dev1 dev2 ana1 mgr1; do sudo chage -d 0 $u; done
116. Verify: for u in dev1 dev2 ana1 mgr1; do echo === $u ===; chage -l $u; done

TASK 4: Disk Quotas for Developers

117. Enable quotas on /home or test filesystem per Section 2.7.
118. Set soft=460800 blocks (450MB), hard=512000 blocks (500MB) for dev1 and dev2.
119. Run: sudo repquota /home > /root/quota_report.txt

TASK 5: Inspection and Documentation

120. Run finger and id for each user. Record the output.

121. Run ls -la on each /srv/novatech directory. Record all permissions.

122. Write a security report covering: users, groups, directory permissions, password policy, quota limits.

Capstone Reflection Questions

1. Which access control model best describes the /srv/novatech structure? Is it DAC, RBAC, or MAC? Justify with evidence.
2. If dev1 tries to access /srv/novatech/management, what exact error appears and why?
3. You need to suspend dev2 temporarily without losing data. What single command is safest?
4. You need to apply 60-day password aging to ALL future users automatically. Which file do you edit?
5. After running groupdel developers, what happens to /srv/novatech/dev_work? What ls -l output do you see?
6. dev2 claims they cannot write to /srv/novatech/dev_work despite apparently correct settings. List three things to check.
7. dev1 has used 495MB and tries to save a 10MB file but gets a 'Disk quota exceeded' error. Explain what is happening.

Chapter 2 — Complete Summary and Quick Reference

This chapter has covered the complete landscape of account management, file permission control, and disk quota administration in Linux. Below is a consolidated reference.

Section	Key Concepts	Essential Commands
2.1 Users & Groups	3 account types (root/human/software); local vs. domain accounts; security vs. distribution groups; AD group scopes (Universal/Global/Domain Local); UPG scheme; umask defaults 022/002/027.	id, groups, umask, cat /etc/group
2.2 Administration	useradd flags (-d -g -m -s -u); usermod (-l rename, -aG safe append, -L/-U lock); userdel (-r removes home); groupadd (-g -o -r -f); groupmod -n; groupdel leaves files.	useradd, usermod, userdel, passwd, groupadd, groupmod, groupdel
2.3 Password Aging	chage (-M max, -m min, -W warn, -I inactive, -E expiry, -l list, -d 0 force reset); /etc/login.defs for system defaults; finger shows login/name/dir/shell/last login.	chage, finger, id, nano /etc/login.defs
2.4 Account Storage	Local vs. LDAP; /etc/passwd 7 fields (username:x:uid:gid:gecos:home:shell); /etc/shadow 8 fields; 'x' in passwd = look in shadow; UID 0 = root always.	cat /etc/passwd, sudo cat /etc/shadow
2.5 File Permissions	UGO model; r(4)/w(2)/x(1); ls -l 10-char string; chmod Method 1 (+/-), Method 2 (=), Method 3 (octal); -R recursive; delete requires directory write permission.	ls -l, chmod
2.6 File Ownership	chown user.group or user:group; chown .group for group-only; -R recursive; chgrp; only root changes user owner; dot vs colon notation.	chown, chgrp
2.7 Disk Quotas	Hard limit (absolute), soft limit (warning), grace period; block quota vs. inode quota; 5 steps: install → /etc/fstab → remount → quotacheck → quotaon/edquota.	quotacheck, quotaon, edquota, quota -u, repquota
Appendix: Access Control	Subject/Object/Action/Policy; DAC (owner-centric, Linux default); RBAC (role-centric, via groups); MAC (system-centric, SELinux/AppArmor); PAM.	chmod/chown (DAC); groups (RBAC)

Must-Know Commands — Complete Quick Reference Card

Command	Purpose	Critical Flag / Note
useradd -d /home/user -g group -m -s /bin/bash user	Create a new user non-interactively.	-m creates home dir; -s sets shell; -g sets primary group.
usermod -aG group user	Add user to supplementary group safely.	-a is MANDATORY. Without -a, all other groups are DESTROYED.
usermod -l newname oldname	Rename a user account.	-l = login name change. Use with -d -m to also move home dir.
usermod -L / -U user	Lock / unlock a user account.	Safer than deletion for temporary suspension.
userdel -r username	Delete user and home directory.	-r removes /home/username and /var/spool/mail/username.
groupadd -g GID groupname	Create a group with a specific GID.	-g specifies GID; -r creates a system group.
groupmod -n newname oldname	Rename an existing group.	-g changes GID; -n changes name.
groupdel groupname	Delete a group.	Files remain — they show numeric GID in ls -l after deletion.
chage -M 90 -m 7 -W 14 user	Set password aging policy.	-l to list; -d 0 forces reset on next login.
finger username	Display user info (login, name, home, shell, last login).	May need: sudo apt install finger
id username	Show UID, GID, and all group memberships.	Always available; no installation needed.
chmod 755 file / chmod u+x file	Set permissions (octal / symbolic).	-R for recursive; = sets exact; + adds; - removes.
chown user:group file	Change user and group owner.	-R recursive; only root can change user owner.
chown .group file	Change only the group owner (dot notation).	Equivalent to: chown :group file
chgrp group file	Change group owner only.	-R for recursive changes.
edquota username	Edit disk quota for a user.	Opens interactive editor for blocks/inodes soft/hard limits.
repquota /path	Report all users' quota usage on a filesystem.	-a for all quota-enabled filesystems.

Top 12 Exam Points for Chapter 2

1. root ALWAYS has UID 0. Privilege comes from the UID, not the name.
2. /etc/shadow is readable ONLY by root — this is why it safely stores password hashes.
3. usermod -G (without -a) DESTROYS all existing group memberships. ALWAYS use -aG.
4. Deleting a file requires WRITE permission on the DIRECTORY, not on the file itself.
5. userdel without -r preserves the home directory. userdel -r deletes everything.
6. groupdel removes the group but NOT the files — files show a numeric GID.
7. Octal chmod: rwx=7, rw-=6, r-x=5, r--=4, -wx=3, -w-=2, --x=1, ---=0.
8. /etc/passwd has 7 fields; /etc/shadow has 8 fields. Both use colon separators.
9. Disk quota hard limit = absolute ceiling. Soft limit = warning threshold with grace period.
10. chown .groupname changes only the group owner (dot before name = group).
11. finger shows login/name/directory/shell/last login. id shows UID/GID/groups.
12. Linux uses DAC (chmod/chown) as its default access control model.